Practical No.: 4

Name: Nikita Lalaso Narute **Roll No.: COB422**

Batch: B2 **Div: B**

Class: BE COMPUTER

Title:

Write a program to solve a 0-1 Knapsack problem using dynamic programming or branch and bound strategy.

Program:

```
#include <bits/stdc++.h>
using namespace std;

int max(int a, int b) {
    return (a > b) ? a : b;
}

int knapSack(int W, int wt[], int val[], int n) {
    if (n == 0 || W == 0)
        return 0;

    if (wt[n - 1] > W)
        return knapSack(W, wt, val, n - 1);

    return max(val[n - 1] +
               knapSack(W - wt[n - 1], wt, val, n - 1),
               knapSack(W, wt, val, n - 1));
}

int main() {
    int profit[] = {70, 90, 10};
    int weight[] = {100, 30, 30};
    int W = 40;
    int n = sizeof(profit) / sizeof(profit[0]);

    cout << knapSack(W, weight, profit, n) << endl;
    return 0;
}
```

Output:

```
(base) sspm@sspm:~$ cd Desktop
(base) sspm@sspm:~/Desktop$ g++ daa4.cpp
(base) sspm@sspm:~/Desktop$ ./a.out
90
(base) sspm@sspm:~/Desktop$
```